

## 4.5. Earthquake Damage in Kavrepalanchok NP

In this assignment, you'll build a classification model to predict building damage for the district of [Kavrepalanchok](#).

```
In [1]: import warnings

warnings.simplefilter(action="ignore", category=FutureWarning)
```

```
In [2]: # Import Libraries here
import sqlite3
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from category_encoders import OneHotEncoder, OrdinalEncoder
from IPython.display import VimeoVideo
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier

warnings.simplefilter(action="ignore", category=FutureWarning)
```

## Prepare Data

### Connect

Run the cell below to connect to the `nepal.sqlite` database.

```
In [ ]: %load_ext sql
%sql sqlite:///../nepal.sqlite
```

**Warning:** Be careful with your SQL queries in this assignment. If you try to get all the rows from a table (for example, `SELECT * FROM id_map`), you will cause an Out of Memory error on your virtual machine. So always include a `LIMIT` when first exploring a database.

#### Task 4.5.1

Write your query in the input box in the left panel.

What's the district ID for Kavrepalanchok? From the lessons, you already know that Gorkha is 4 ; from the textbook, you know that Ramechhap is 2 . Of the remaining districts, Kavrepalanchok is the one with the largest number of observations in the `id_map` table.

#### Task 4.5.2

Write your query in the input box in the left panel.

#### Task 4.5.3

Write your query in the input box in the left panel.

#### Task 4.5.4

Write your query in the input box in the left panel.

## Import

#### Task 4.5.5

```
In [3]: def wrangle(db_path):
# Connect to database
conn = sqlite3.connect(db_path)

# Construct query
query = """
    SELECT distinct(i.building_id) AS b_id,
           s.*,
           d.damage_grade
    FROM id_map AS i
    JOIN building_structure AS s ON i.building_id = s.building_id
    JOIN building_damage AS d ON i.building_id = d.building_id
    WHERE i.district_id = 3
    """

# Read query results into DataFrame
df = pd.read_sql(query, conn, index_col="b_id")

# Identify Leaky columns (post-earthquake info)
drop_cols = [col for col in df.columns if "post_eq" in col]

# Create binary target column
df["damage_grade"] = df["damage_grade"].str[-1].astype(int)
df["severe_damage"] = (df["damage_grade"] > 3).astype(int)

# Drop old target
drop_cols.append("damage_grade")

# Drop multicollinearity column
```

```

drop_cols.append("count_floors_pre_eq")

# Drop high-cardinality / unique ID column
drop_cols.append("building_id")

df.drop(columns=drop_cols, inplace=True)

return df

```

Use your `wrangle` function to query the database at `"../nepal.sqlite"` and return your cleaned results.

```

In [4]: df = wrangle("../nepal.sqlite")
df.head()

```

```

Out[4]:      age_building  plinth_area_sq_ft  height_ft_pre_eq  land_surface_condition  founda
b_id

```

|              |    |     |    |      |          |
|--------------|----|-----|----|------|----------|
| <b>87473</b> | 15 | 382 | 18 | Flat | Mu<br>St |
| <b>87479</b> | 12 | 328 | 7  | Flat | Mu<br>St |
| <b>87482</b> | 23 | 427 | 20 | Flat | Mu<br>St |
| <b>87491</b> | 12 | 427 | 14 | Flat | Mu<br>St |
| <b>87496</b> | 32 | 360 | 18 | Flat | Mu<br>St |



## Explore

### Slight Code Change

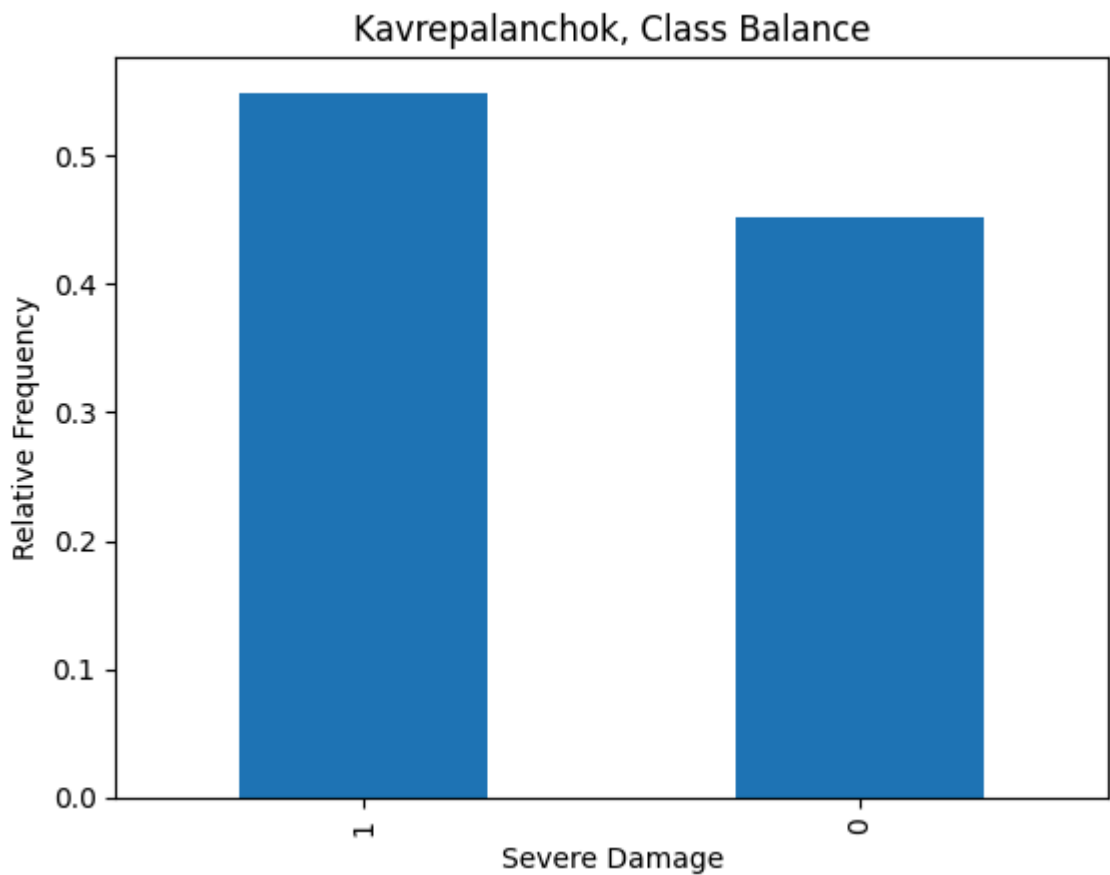
In the following task, you'll notice a small change in how plots are created compared to what you saw in the lessons. While the lessons use the global matplotlib method like `plt.plot(...)`, in this task, you are expected to use the object-oriented (OOP) API instead. This means creating your plots using `fig, ax = plt.subplots()` and then calling plotting methods on the `ax` object, such as `ax.plot(...)`, `ax.hist(...)`, or `ax.scatter(...)`.

If you're using pandas' or seaborn's built-in plotting methods (like `df.plot()` or `sns.lineplot()`), make sure to pass the `ax=ax` argument so that the plot is rendered on the correct axes.

This approach is considered best practice and will be used consistently across all graded tasks that involve matplotlib.

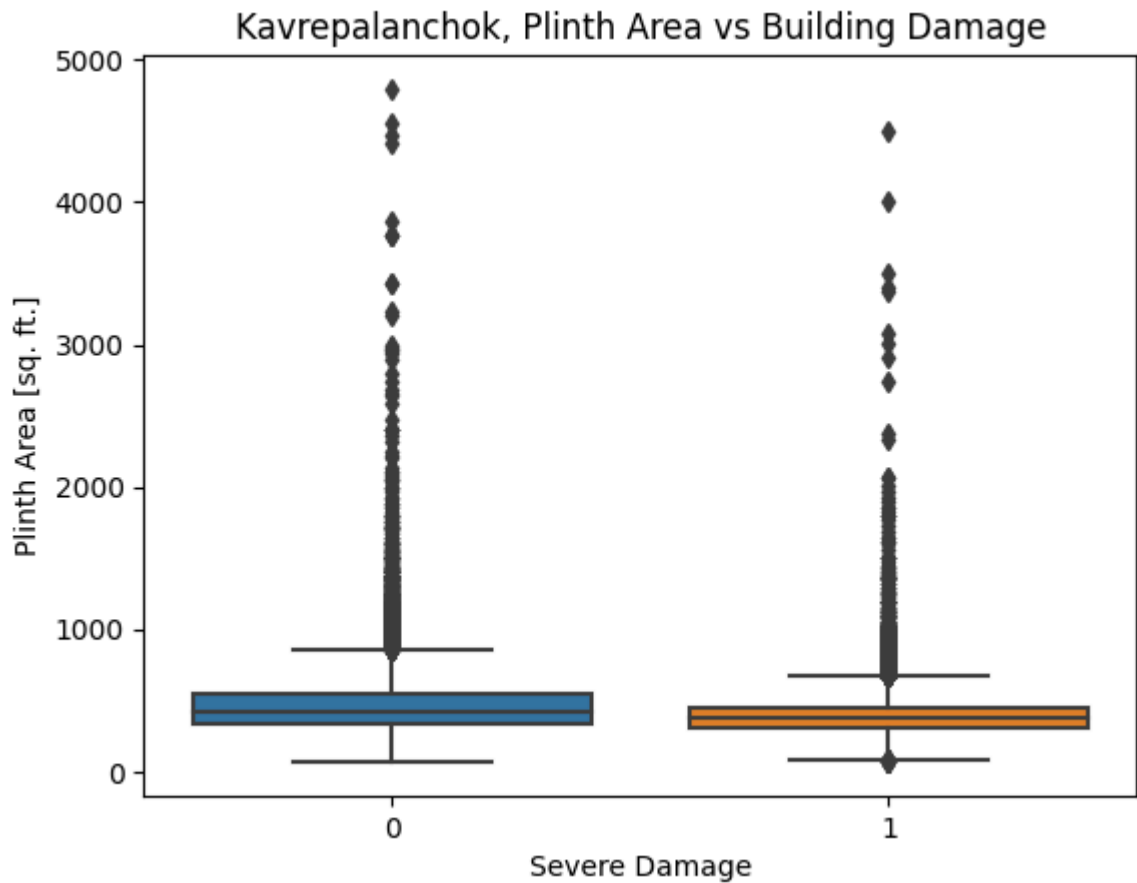
### Task 4.5.6

```
In [7]: # Plot value counts of "severe_damage"
fig, ax = plt.subplots()
df["severe_damage"].value_counts(normalize=True).plot(
    kind="bar",
    xlabel="Severe Damage",
    ylabel="Relative Frequency",
    title="Kavrepalanchok, Class Balance",
    ax=ax
);
```



### Task 4.5.7

```
In [9]: fig, ax = plt.subplots()
sns.boxplot(x="severe_damage", y="plinth_area_sq_ft", data=df, ax=ax)
ax.set_xlabel("Severe Damage")
ax.set_ylabel("Plinth Area [sq. ft.]")
ax.set_title("Kavrepalanchok, Plinth Area vs Building Damage");
```



#### Task 4.5.8

```
In [50]: roof_pivot = pd.pivot_table(
          df, index="roof_type", values="severe_damage", aggfunc=np.mean
        )
          roof_pivot
```

```
Out[50]:
```

|                          | severe_damage |
|--------------------------|---------------|
| roof_type                |               |
| Bamboo/Timber-Heavy roof | 0.569477      |
| Bamboo/Timber-Light roof | 0.604842      |
| RCC/RB/RBC               | 0.040715      |

## Split

#### Task 4.5.9

```
In [15]: target = "severe_damage"
          X = df.drop(columns=target)
          y = df[target]
```

#### Task 4.5.10

```
In [18]: X_train, X_val, y_train, y_val = train_test_split(
          X, y, test_size=0.2, random_state=42
```

```
)
```

# Build Model

## Baseline

### Task 4.5.11

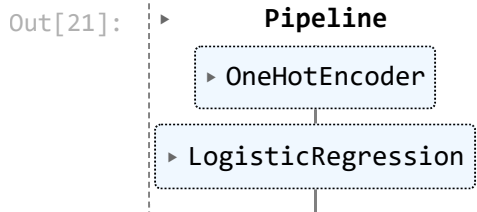
```
In [20]: acc_baseline = y_train.value_counts(normalize=True).max()  
print("Baseline Accuracy:", round(acc_baseline, 2))
```

Baseline Accuracy: 0.55

## Iterate

### Task 4.5.12

```
In [21]: model_lr = make_pipeline(  
    OneHotEncoder(use_cat_names=True),  
    LogisticRegression(max_iter=1000)  
)  
model_lr.fit(X_train, y_train)
```



### Task 4.5.13

```
In [23]: lr_train_acc = accuracy_score(y_train, model_lr.predict(X_train))  
lr_val_acc = accuracy_score(y_val, model_lr.predict(X_val))  
  
print("Logistic Regression, Training Accuracy Score:", lr_train_acc)  
print("Logistic Regression, Validation Accuracy Score:", lr_val_acc)
```

Logistic Regression, Training Accuracy Score: 0.6509489432593996  
Logistic Regression, Validation Accuracy Score: 0.6521852747109166

### Task 4.5.14

```
In [24]: depth_hyperparams = range(1, 16)  
training_acc = []  
validation_acc = []  
  
for d in depth_hyperparams:  
    model_dt = make_pipeline(  
        OrdinalEncoder(),  
        DecisionTreeClassifier(max_depth=d, random_state=42)  
    )  
    model_dt.fit(X_train, y_train)
```

```

training_acc.append(model_dt.score(X_train, y_train))
validation_acc.append(model_dt.score(X_val, y_val))

print("Training Accuracy Scores:", training_acc[:3])
print("Validation Accuracy Scores:", validation_acc[:3])

```

Training Accuracy Scores: [0.6303041191650606, 0.6303041191650606, 0.642292490118577]

Validation Accuracy Scores: [0.6350035931273273, 0.6350035931273273, 0.6453909975828053]

```

In [26]: # Run this cell
submission = pd.Series(validation_acc, index=depth_hyperparams)
submission

```

```

Out[26]: 1      0.635004
2      0.635004
3      0.645391
4      0.652773
5      0.652904
6      0.658457
7      0.658457
8      0.662769
9      0.664533
10     0.665382
11     0.662899
12     0.660613
13     0.655321
14     0.655648
15     0.654733
dtype: float64

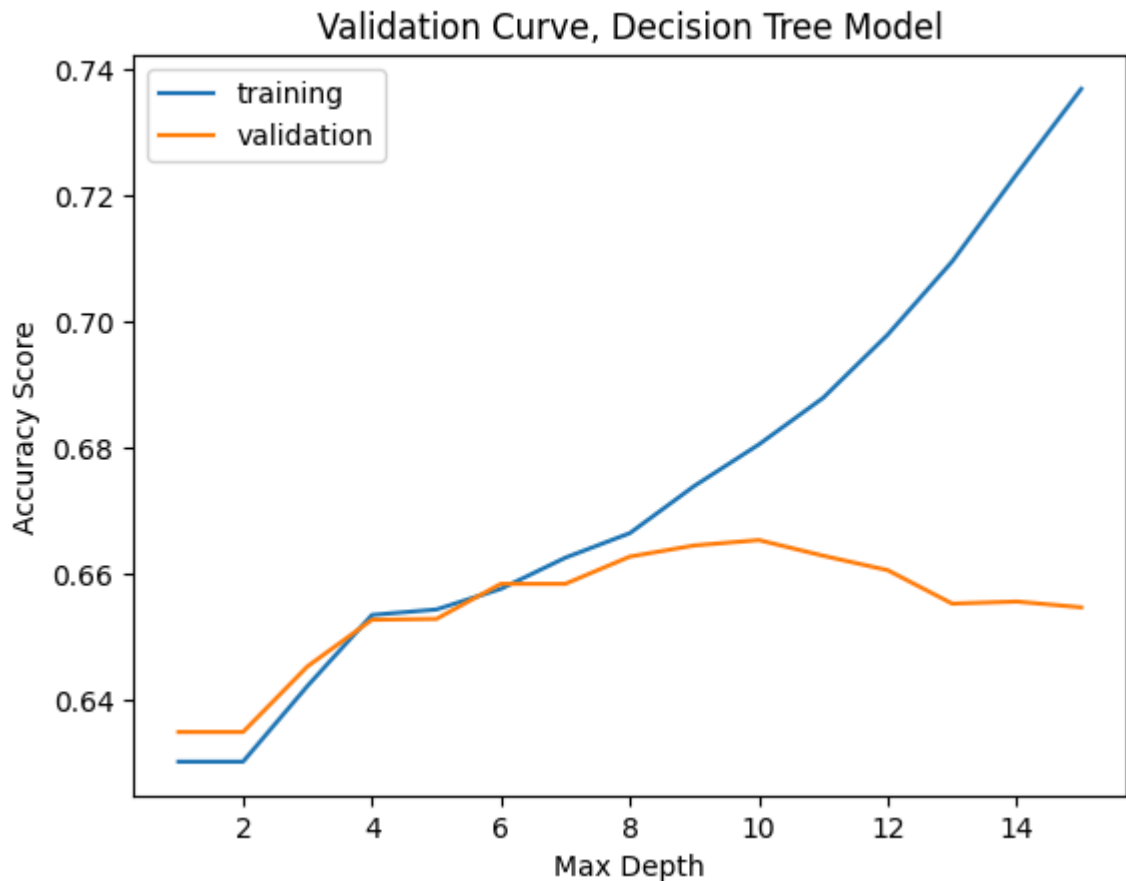
```

#### Task 4.5.15

```

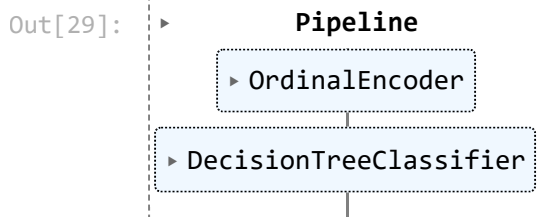
In [28]: fig, ax = plt.subplots()
ax.plot(depth_hyperparams, training_acc, label="training")
ax.plot(depth_hyperparams, validation_acc, label="validation")
ax.set_xlabel("Max Depth")
ax.set_ylabel("Accuracy Score")
ax.set_title("Validation Curve, Decision Tree Model")
ax.legend();

```



#### Task 4.5.16

```
In [29]: final_model_dt = make_pipeline(  
    OrdinalEncoder(),  
    DecisionTreeClassifier(max_depth=10, random_state=42)  
)  
final_model_dt.fit(X_train, y_train)
```



## Evaluate

#### Task 4.5.17

```
In [47]: X_test = pd.read_csv("data/kavrepalanchok-test-features.csv", index_col="b_id")  
y_test_pred = final_model_dt.predict(X_test)  
y_test_pred[:5]
```

Out[47]: array([1, 1, 1, 1, 0])

## Communicate Results

### Task 4.5.18

```
In [40]: features = X_train.columns
importances = final_model_dt.named_steps["decisiontreeclassifier"].feature_importances_
feat_imp = pd.Series(importances, index=features).sort_values(kind="stable")
feat_imp
```

```
Out[40]: plan_configuration      0.004189
land_surface_condition      0.008599
foundation_type              0.009967
position                    0.011795
ground_floor_type           0.013521
other_floor_type            0.025336
plinth_area_sq_ft           0.073128
superstructure              0.082698
age_building                0.112641
height_ft_pre_eq            0.130728
roof_type                   0.527399
dtype: float64
```

### Task 4.5.19

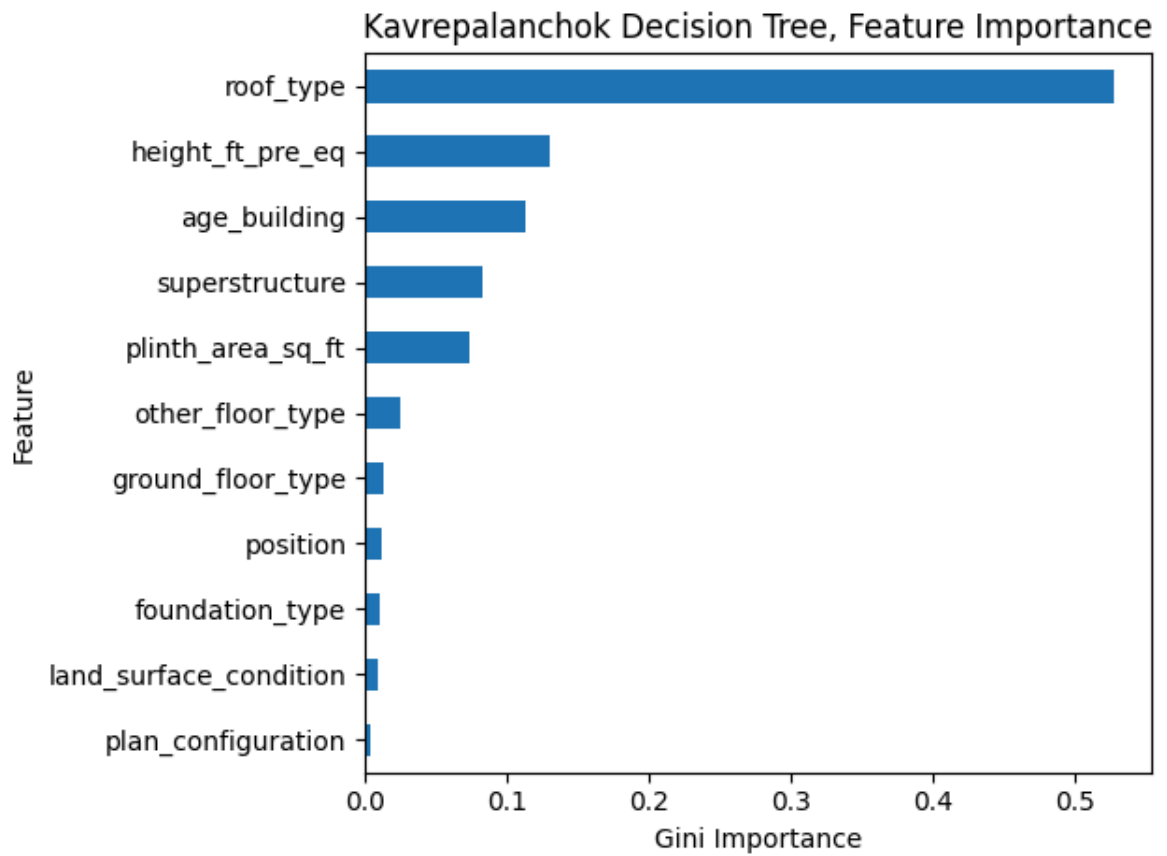
Do you see any relationship between this plot and the exploratory data analysis you did regarding roof type?

```
In [36]: fig, ax = plt.subplots()

# Create the horizontal bar plot on the axes object
feat_imp.plot(kind="barh", ax=ax)

# Set labels and title
ax.set_xlabel("Gini Importance")
ax.set_ylabel("Feature")
ax.set_title("Kavrepalanchok Decision Tree, Feature Importance")

# Apply tight layout
fig.tight_layout()
```



Congratulations! You made it to the end of Project 4. 🎉🎉🎉

---

Copyright 2023 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.